

Jaya Stores E-commerce Platform Documentation

Author: Kishore Balaji P

Date: July 2025

Contents

Table of Contents

1. Abstract / Executive Summary
2. Objectives and Scope
3. System Architecture
4. Tech Stack Overview
5. Features Overview
 - 5.1 User-Facing Features
 - 5.2 Admin Panel Features
6. Folder Structure
7. Key API Endpoints
8. Deployment Overview
9. Implementation & Contributions
10. Challenges & Solutions
11. Future Enhancements
12. Conclusion
13. References / Sources

Abstract / Executive Summary

Jaya Stores is a full-stack online retail platform developed to serve the e-commerce needs of my village. The platform consists of a React and TypeScript frontend hosted on Vercel and a Node.js and Express backend connected to a PostgreSQL (Neon DB) database. Product images are stored using AWS S3.

The backend APIs are deployed on AWS Lambda using a serverless architecture for scalability. Key features include product browsing, user registration with OTP verification, shopping cart functionality, Cash on Delivery (COD), and a chatbot to assist users. Payment gateway integration was attempted but is currently inactive due to account verification issues. The platform is designed for local usage and is expected to generate a profit of approximately ₹50,000.

Objectives and Scope

Objectives

- **User Shopping Experience:**
Provide a smooth online shopping experience with user registration, product browsing, cart management, and checkout. Registration requires OTP verification after accepting terms and conditions.
- **Admin Management Tools:**
Enable administrators to manage users, products, and orders through a dashboard that displays key metrics such as users, orders, and revenue.
- **Modern Tech Stack:**
Use modern and maintainable technologies such as React, Tailwind CSS, Node.js, Express, and PostgreSQL.
- **Scalability & Reliability:**
Deploy backend APIs on AWS Lambda to support automatic scaling without managing servers.

- **Security & Compliance:**

Ensure secure authentication using hashed passwords, JWT tokens, and HTTPS.

Scope

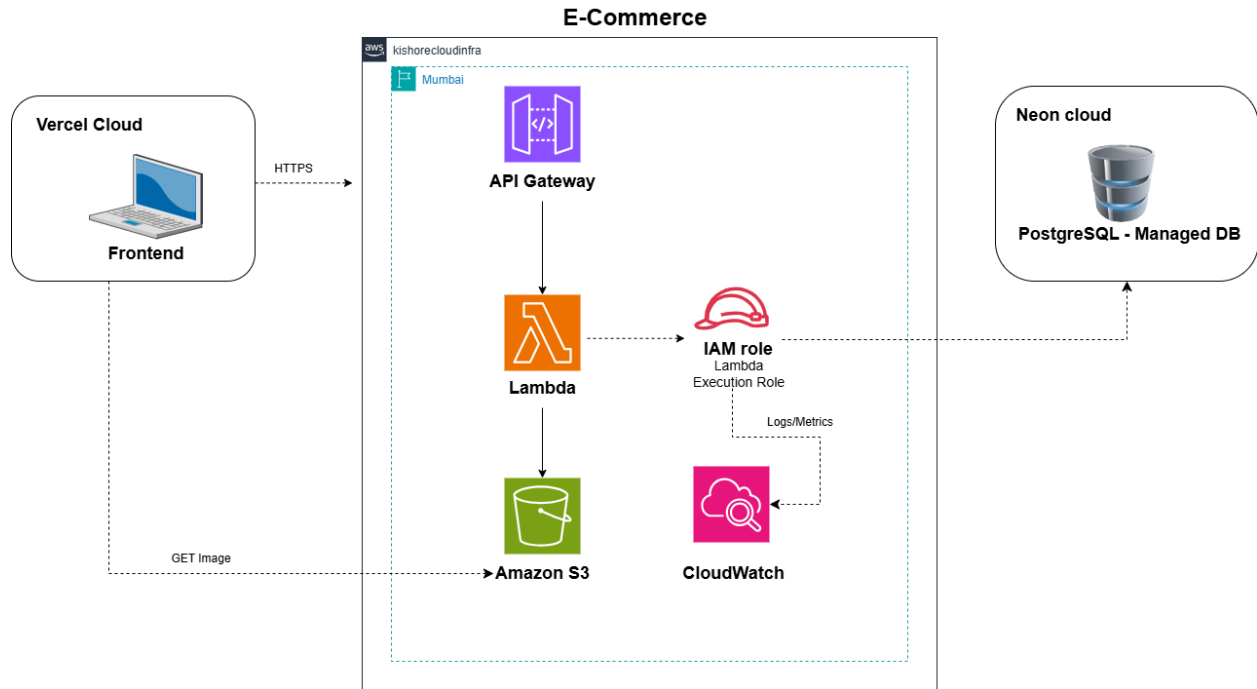
The project includes user features such as product browsing, shopping, checkout, and order history, along with admin features for managing products, users, and orders. The system is deployed using cloud services to ensure scalability and reliability.

System Architecture

The Jaya Stores application follows a modular, cloud-native architecture. Users access the platform through a web browser or mobile device, which interacts with the React and TypeScript frontend hosted on Vercel. The frontend is responsible for rendering the user interface and sending secure HTTPS requests for all backend operations.

These requests are routed through AWS API Gateway to AWS Lambda functions, where core business logic such as user authentication, product management, cart handling, and order processing is executed. Lambda uploads product images to Amazon S3 using its execution role and stores the generated image URLs in the Neon PostgreSQL database. When products are displayed, the browser loads images directly from S3, reducing backend workload. AWS CloudWatch captures logs and metrics from Lambda for monitoring and debugging.

This architecture enables automatic scaling without server management, improves performance through direct image delivery, and ensures clear separation between presentation, business logic, and data storage. Overall, the design provides a secure, scalable, and maintainable foundation suitable for a real-world e-commerce platform.



Tech Stack Overview

The project uses a modern and maintainable technology stack.

Frontend

- React (TypeScript)
- Tailwind CSS
- Vite
- Axios
- React Router

Backend

- Node.js
- Express.js
- PostgreSQL (Neon DB)
- AWS Lambda

- AWS S3
- JWT Authentication

Deployment Tools

- Vercel
- AWS API Gateway
- GitHub Actions
- AWS CloudWatch

Features Overview

User-Facing Features

- **Account Management:**
Users can register, log in, and manage their profiles. OTP verification is required during registration.
- **Product Catalog:**
Users can browse and search products. Each product displays images, price, description, and stock availability.
- **Shopping Cart:**
Users can add and remove items from their cart, which is stored on the backend for logged-in users.
- **Checkout & Payments:**
Cash on Delivery (COD) is supported. Payment gateway integration was attempted but is currently inactive.
- **Order History:**
Users can view past orders and order status.

- **Chatbot Service:**
A chatbot assists users with navigation and common queries.
- **Responsive Design:**
The interface is responsive for both desktop and mobile devices.

Admin Panel Features

- Secure admin authentication
- Product management (add, update, delete)
- Order management and status updates
- User management
- Basic analytics and statistics

Folder Structure

Frontend

- components/ – Reusable UI components
- pages/ – User and admin pages
- context/ – Authentication and cart state
- services/ – API communication
- types/ – TypeScript interfaces

Backend

- routes/ – API routes
- middleware/ – Authentication and admin checks
- utils/ – Email and AWS S3 helpers
- db.js – Database connection
- deploy-lambda.yml – CI/CD workflow

This structure separates concerns and keeps the codebase organized and maintainable.

Key API Endpoints

Authentication

- POST /api/auth/register – User registration
- POST /api/auth/login – User login

Products

- GET /api/products – List products
- GET /api/products/:id – Product details

Cart & Orders

- POST /api/cart – Add to cart
- GET /api/cart – View cart
- POST /api/orders – Create order
- GET /api/orders – View orders

Admin

- POST /api/admin/products – Add product
- PUT /api/admin/products/:id – Update product
- DELETE /api/admin/products/:id – Delete product
- GET /api/admin/orders – View all orders

Deployment Overview

- **Backend:** AWS Lambda with API Gateway
- **Database:** Neon DB (PostgreSQL)
- **Frontend:** Vercel
- **Images:** AWS S3
- **CI/CD:** GitHub Actions
- **Monitoring:** AWS CloudWatch

This setup ensures scalability, reliability, and cost efficiency.

Implementation & Contributions

As the sole developer of Jaya Stores, I:

- Designed and implemented backend APIs
- Built the frontend using React and Tailwind CSS
- Designed the database schema
- Configured AWS Lambda, API Gateway, and S3
- Deployed the frontend on Vercel
- Implemented JWT authentication and OTP verification
- Set up CI/CD using GitHub Actions
- DevOps & Testing: Set up a CI/CD pipeline using GitHub Actions to run tests and deploy backend updates automatically to Lambda. Wrote unit and integration tests for both backend APIs and frontend components to ensure reliability.

Challenges & Solutions

During development, a few technical challenges were encountered while building and deploying the application.

- **Authentication Security:** Implemented JWT and bcrypt
- **CORS Issues:** Resolved through proper backend configuration
- **Serverless Deployment:** Optimized Lambda bundle size
- **UI Design:** Achieved consistency using Tailwind CSS

Future Enhancements

- Enable live PhonePe payment gateway
- Add product reviews and ratings
- Improve admin analytics with charts
- Develop a mobile app or PWA
- Add multi-language support

Conclusion

Jaya Stores is a complete e-commerce platform built using modern web and cloud technologies. It provides a full user journey from browsing products to checkout, along with an admin system for managing the store. The serverless architecture ensures scalability and cost efficiency, and the platform forms a strong base for future enhancements.

Sources

1. Global Ecommerce Sales Growth Report (2025) - Shopify
<https://www.shopify.com/blog/global-ecommerce-sales>
2. React | Meta Open Source
<https://opensource.fb.com/projects/react/>
3. Tailwind CSS - Rapidly build modern websites
<https://tailwindcss.com/>
4. Node.js Introduction | The Odin Project
<https://www.theodinproject.com/lessons/nodejs-introduction-what-is-nodejs>
5. Vite Guide
<https://vite.dev/guide/>
6. AWS Guidance for Web Store
<https://d1.awsstatic.com/architecture-diagrams/ArchitectureDiagrams/guidance-for-web-store-on-aws-ra.pdf>
7. PostgreSQL: About
<https://www.postgresql.org/about/>
8. Top E-commerce Website Features – Admin Panel
<https://seclgroup.com/top-ecommerce-website-features-part-2/>
9. Web API Design Best Practices – Microsoft Learn
<https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design>
10. A Guide to Developing Serverless E-commerce Workflows | AWS
<https://aws.amazon.com/blogs/industries/a-guide-to-developing-serverless-ecommerce-workflows-for-commercetools/>